

Demystifying the most significant Java 9 language features



Krakow, 9-11 May 2018

Ionuț Baloșin

Software Architect
 @ionutbalosin

About Me

Ionuț Baloșin

Software Architect @ Luxoft

Technical Trainer

- Java Performance and Tuning
- Software Architecture



www.ionutbalosin.com
 [@ionutbalosin](https://twitter.com/ionutbalosin)

To demystify - to make
something easier to understand

dictionary.cambridge.org

To demystify - to make
something easier to understand

dictionary.cambridge.org

My definition - understand Java language features by
digging into JVM internals (i.e. bytecode, assembly)

“Explore the world. Nearly everything is really interesting if you go into it deeply enough.”



Richard Feynman

Agenda

Private interface methods

String in Java 9

StackWalker

Collection factory methods

onSpinWait()

Contended locks in Java 9

My Configuration

Hardware

- Intel i7-6700HQ Skylake
- 16GB DDR4 2133 MHz



Software

- Ubuntu 16.04.2
- JDK 9.0.4 x64
- Java Microbenchmark Harness
- Eclipse Memory Analyzer



Private interface methods

JEP 213: Milling Project Coin

Author Joseph D. Darcy

Owner Joe Darcy

Status Closed / Delivered

Reviewed by Brian Goetz

Endorsed by Brian Goetz

Release 9

Issue [8042880](#)

Summary

The small language changes included in Project Coin / [JSR 334](#) as part of JDK 7 / Java SE 7 have been easy to use and have worked well in practice. However, a few amendments could address the rough edges of those changes. In addition, using underscore ("_") as an identifier, which generates a warning as of Java SE 8, should be turned into an error in Java SE 9. It is also proposed that interfaces be allowed to have private methods.

Description

Support for private methods in interfaces was briefly in consideration for inclusion in Java SE 8 as part of the effort to add support for Lambda Expressions, but was withdrawn to enable better focus on higher priority tasks for Java SE 8. [It is now proposed that support for private interface methods be undertaken](#) thereby enabling non abstract methods of an interface to share code between them.

```
public interface IPrivateMethod {  
  
    default void foo() {  
        print();  
    }  
  
    private void print() {  
        System.out.println("Private Method");  
    }  
}
```

```

public interface IPrivateMethod {

    default void foo() {
        print();
    }

    private void print() {
        System.out.println("Private Method");
    }
}

```



```

public interface IPrivateMethod {

    public void foo();
    Code:
        0: aload 0
        1: invokespecial #2          // InterfaceMethod print:()V
        4: return

    private void print();
    //...
}

```

`private` interface method is called using `INVOKESPECIAL`, as any private class method

`default` is a keyword only at Java Language level, it doesn't exist at bytecode level

```
public interface IPrivateMethod {  
  
    static void baz() {  
        staticPrint();  
    }  
  
    private static void staticPrint() {  
        System.out.println("Private Static Method");  
    }  
}
```

```
public interface IPrivateMethod {

    static void baz() {
        staticPrint();
    }

    private static void staticPrint() {
        System.out.println("Private Static Method");
    }
}
```



```
public interface IPrivateMethod {

    public static void baz();
    Code:
        0: invokestatic #1          // InterfaceMethod staticPrint:()V
        3: return

    private static void staticPrint();
    // ...

}
```

`private static` interface method is called using `INVOKESTATIC`, as any static class method

A more 'interesting' example

e.g. anonymous class casted to a functional interface using a private method reference

```
@FunctionalInterface
public interface IFunctionalInterface {

    void run();
}
```

```
@FunctionalInterface
public interface IFunctionalInterface {

    void run();
}

public class Main {

    public static void main(String[] args) {
        IFunctionalInterface test = ((Inner) (new Inner() {}))::print;
        test.run();
    }

    public interface Inner {
        private void print() {
            System.out.println("Private Method");
        }
    }
}
```

```
@FunctionalInterface
public interface IFunctionalInterface {

    void run();
}

public class Main {

    public static void main(String[] args) {
        IFunctionalInterface test = ((Inner) (new Inner() {}))::print;
        test.run();
    }
}

public interface Inner {
    private void print() {
        System.out.println("Private Method");
    }
}
```

Anonymous class casted to a functional interface using a private method reference

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```



nested anonymous class

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

nested anonymous class

new Inner()

{
new #2
dup
invokespecial #3

// class **Main\$1**

// Method **Main\$1."<init>": ()V**

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

nested anonymous class

new Inner()

new #2
dup
invokespecial #3

// class **Main\$1**

// Method **Main\$1."<init>": ()V**

```
class Main$1 implements Main$Inner {  
    // default constructor  
}
```

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

private method reference



```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

```
INVOKEDYNAMIC run(LMain$Inner;)LFunctionalInterface; [  
  // handle kind 0x6 : INVOKESTATIC  
  java/lang/invoke/LambdaMetafactory.metafactory(..)Ljava/lang/invoke/CallSite;  
  // arguments:  
  ()V,  
  // handle kind 0x6 : INVOKESTATIC  
  Main.lambda$main$0(LMain$Inner;)V,  
  ()V  
]
```

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

```
INVOKEDYNAMIC run(LMain$Inner;) LIFunctionalInterface; [  
  // handle kind 0x6 : INVOKESTATIC  
  java/lang/invoke/LambdaMetafactory.metafactory(..) Ljava/lang/invoke/CallSite;  
  // arguments:  
  ()V,  
  // handle kind 0x6 : INVOKESTATIC  
  Main.lambda$main$0(LMain$Inner;) V,  
  ()V
```

]

synthetic method generated by javac

```
IFunctionalInterface test = ((Inner)(new Inner() {}))::print;
test.run();
```

```
INVOKEDYNAMIC run(LMain$Inner;)V,
// handle kind 0x00000000: INVOKEDYNAMIC
java/lang/invoke/MethodHandleImpl::run
// arguments:
()V,
// handle kind 0x00000000: INVOKEDYNAMIC
Main.lambda$main$0(LMain$Inner;)V,
()V
```



```
public class Main {
    // ...
    private static synthetic lambda$main$0(Main$Inner var1) {
        Main$Inner.access$000(var1)
    }
}
```

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

Returns an instance of the functional interface
i.e. VM generated anonymous class **Main\$\$Lambda\$49**



```
INVOKEDYNAMIC run(LMain$Inner;) LFunctionalInterface; [  
  // handle kind 0x6 : INVOKESTATIC  
  java/lang/invoke/LambdaMetafactory.metafactory(..) Ljava/lang/invoke/CallSite;  
  // arguments:  
  ()V,  
  // handle kind 0x6 : INVOKESTATIC  
  Main.lambda$main$0(LMain$Inner;) V,  
  ()V  
]
```

**How does the VM auto-generated
Main\$\$Lambda\$49 class look?**

-Djdk.internal.lambda.dumpProxyClasses=<path>

```
final class Main$$Lambda$49 implements IFunctionalInterface {
    private final Main$Inner arg$1;

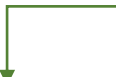
    private Main$$Lambda$49(Main$Inner var1) {
        this.arg$1 = var1;
    }

    private static IFunctionalInterface get$Lambda(Main$Inner var0) {
        return new Main$$Lambda$49(var0);
    }

    @Hidden
    public void run() {
        Main.lambda$main$0(this.arg$1);
    }
}
```

-Djdk.internal.lambda.dumpProxyClasses=<path>

Global **LambdaMetafactory** counter used to generate unique VM anonymous class names



```
final class Main$$Lambda$49 implements IFunctionalInterface {
    private final Main$Inner arg$1;

    private Main$$Lambda$49(Main$Inner var1) {
        this.arg$1 = var1;
    }

    private static IFunctionalInterface get$Lambda(Main$Inner var0) {
        return new Main$$Lambda$49(var0);
    }

    @Hidden
    public void run() {
        Main.lambda$main$0(this.arg$1);
    }
}
```

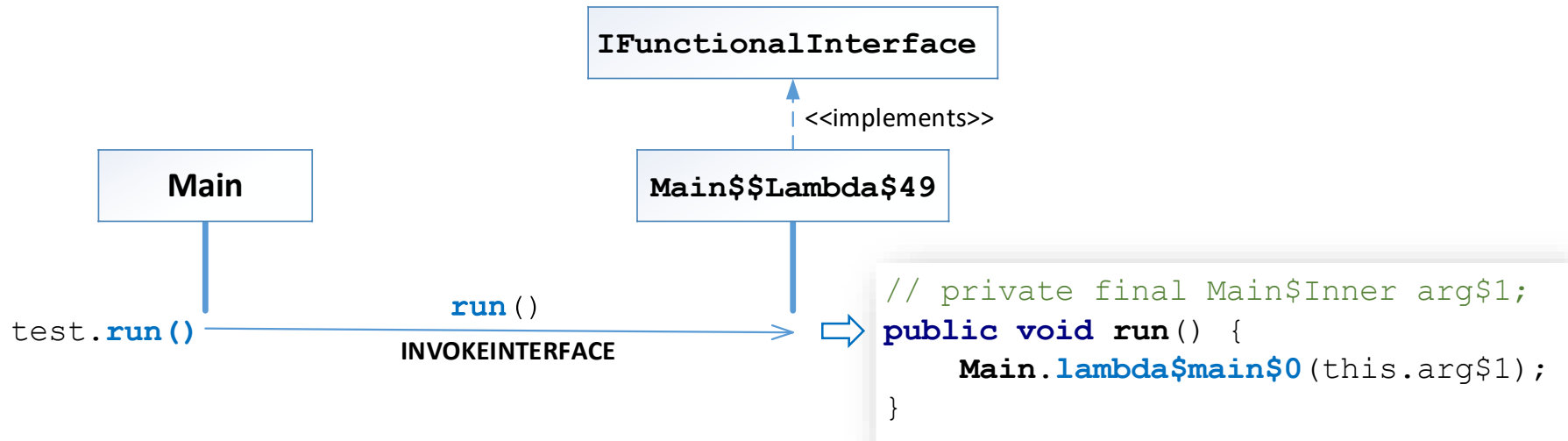
-Djdk.internal.lambda.dumpProxyClasses=<path>

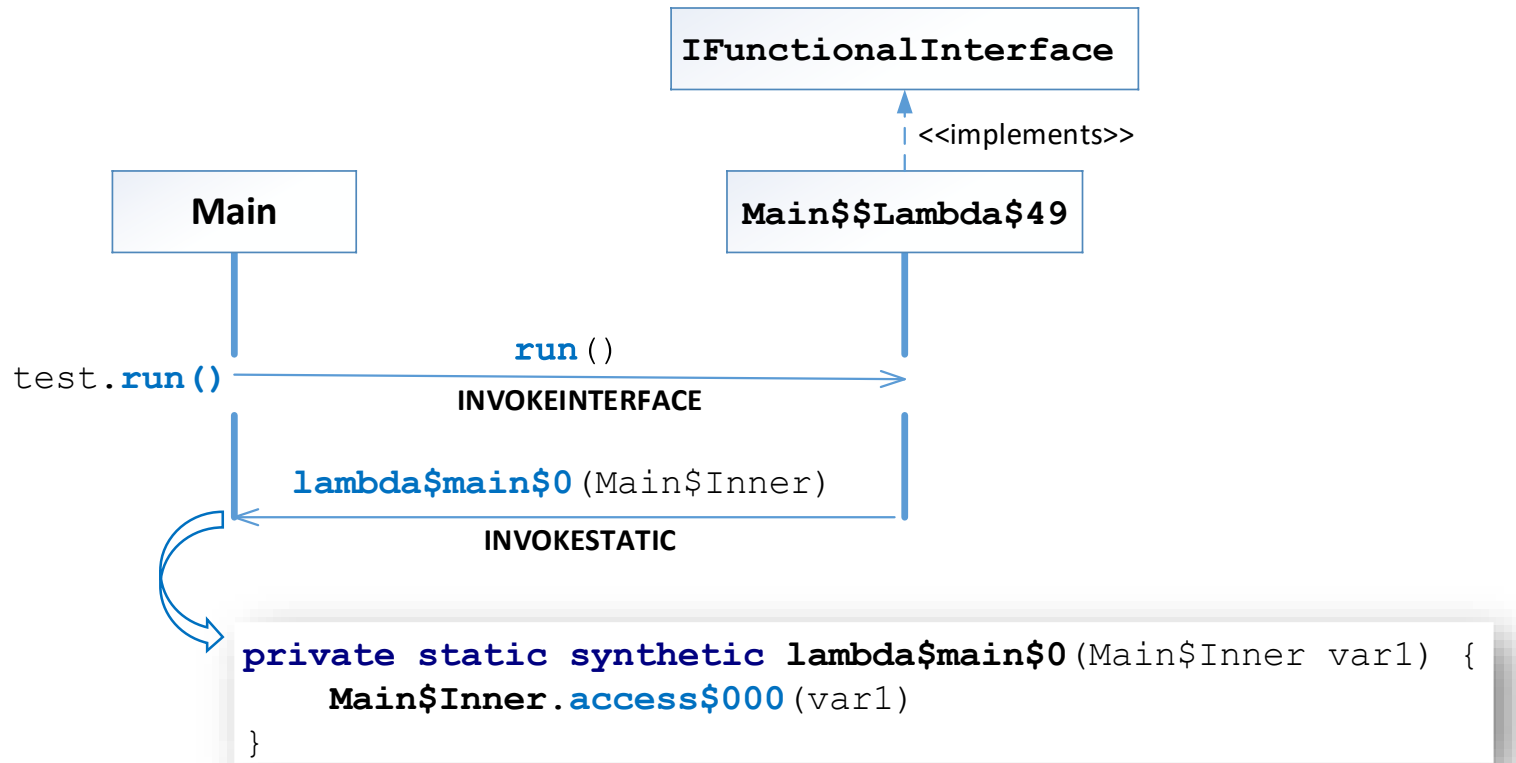
```
final class Main$$Lambda$49 implements IFunctionalInterface {  
    private final Main$Inner arg$1;  
  
    private Main$$Lambda$49(Main$Inner var1) {  
        this.arg$1 = var1;  
    }  
  
    private static IFunctionalInterface get$Lambda(Main$Inner var0) {  
        return new Main$$Lambda$49(var0);  
    }  
  
    @Hidden  
    public void run() {  
        Main.lambda$main$0(this.arg$1);  
    }  
}
```

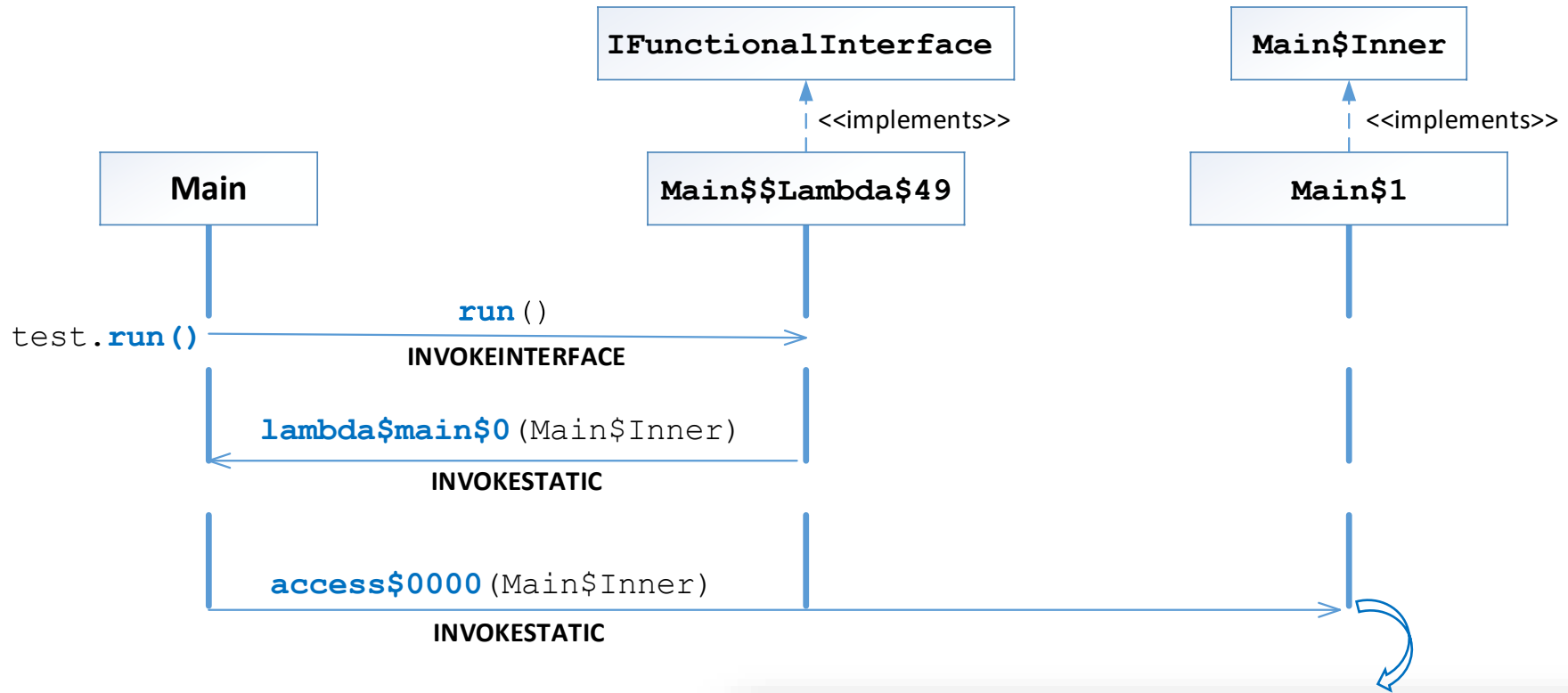
calls method defined in **Main**

```
IFunctionalInterface test = ((Inner) (new Inner() {}))::print;  
test.run();
```

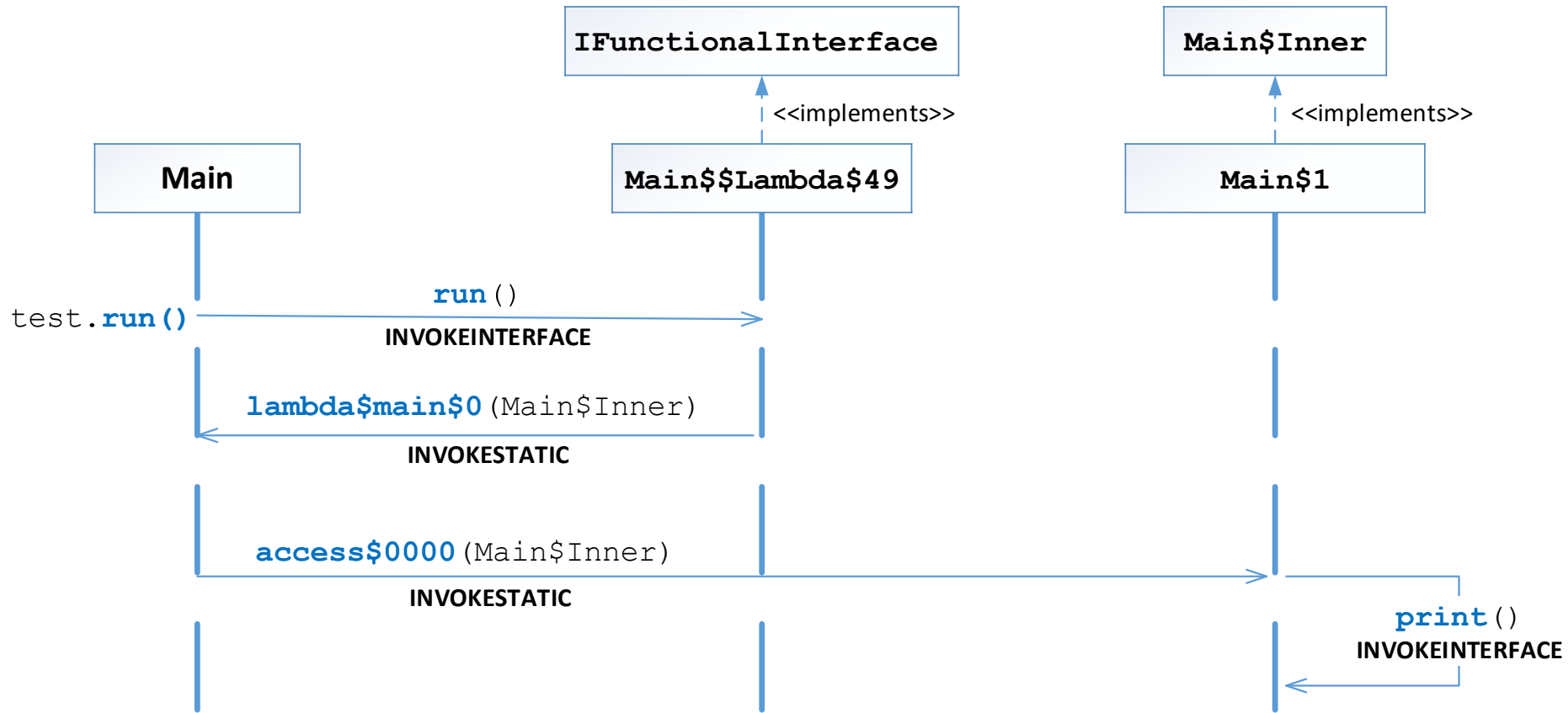
```
aload_1  
invokeinterface #6, 1          // InterfaceMethod IFunctionalInterface.run:()V
```







```
public static void access$000 (Main$Inner var0) {  
    var0.print();  
}
```



In a nutshell:

**few generated classes w/ synthetic
methods for dispatching method
calls!**

but ... what about the next scenario?



Will this code compile? (i.e. remove explicit **Inner** cast)

```
public class Main {  
  
    public static void main(String[] args) {  
        IFunctionalInterface test = ((Inner)(new Inner() {}))::print;  
        test.run();  
    }  
  
    public interface Inner {  
        private void print() {  
            System.out.println("Private Method");  
        }  
    }  
}
```



Will this code compile? (i.e. remove explicit **Inner** cast)

```
public class Main {  
  
    public static void main(String[] args) {  
        IFunctionalInterface test = (Inner (new Inner() {}))::print;  
        test.run();  
    }  
  
    public interface Inner {  
        private void print() {  
            System.out.println("Private Method");  
        }  
    }  
}
```

error: print() has private access in Main.Inner



JDK / JDK-8194998

Interface with private method can work as a functional interface

Details

Type: Bug

Status:

IN PROGRESS

Priority: P3

Resolution:

Unresolved

Affects Version/s: 9.0.4, 10

Fix Version/s:

11

Description

A DESCRIPTION OF THE PROBLEM :

The following code is quite confusing, as after casting an anonymous class to its interface it can work as a functional interface (in various constructions actually):

```
public class A {  
    public static void main(String[] args) {  
        Runnable test1 = ((I)(new I() {})).test; // compiles OK  
        Runnable test2 = ((new I() {})).test; // won't compile  
    }  
    interface I {  
        private void test() {}  
    }  
}
```

✓ Vicente Arturo Romero Zaldivar added a comment - 2018-01-12 08:36

evaluation from Maurizio:

new I() {} doesn't have same members as I, the latter has a 'test' method but, since test is private it is not inherited by a subclass so the compiler is right in rejecting new I() {}::test but the diagnostic is wrong

String in Java 9

JEP 254: Compact Strings

```
public final class String {  
  
    private final byte[] value;  
    private final byte coder;  
    private int hash;  
  
}
```

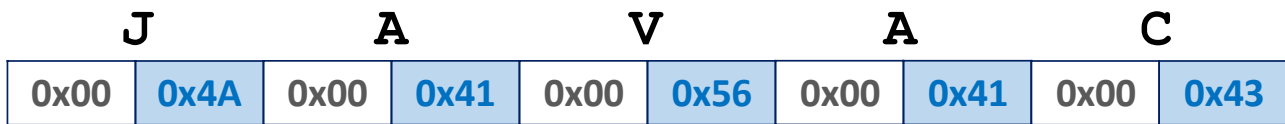
```
coder == LATIN1 || UTF16  
    // coder - indicates which encoding is used
```

```
static final byte LATIN1 = 0;  
static final byte UTF16  = 1;
```

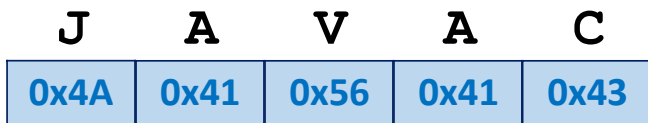
-XX:+CompactStrings - enabled by default

```
String javaString = new String("JAVAC");
```

UTF-16



LATIN-1



1 byte

```
String javaString = new String("JAVAC");
```

UTF-16

Class Name	Shallow Heap	Retained Heap
<Regex>	<Numeric>	<Numeric>
▼ javaString java.lang.String @ 0x6d21347b0	24	56
> <class> class java.lang.String @ 0x6d2700980 System Class, Native Stack	24	56
> value byte[10] @ 0x6d21347e0 J.A.V.A.C.	32	32
Σ Total: 2 entries		

LATIN-1

Class Name	Shallow Heap	Retained Heap
<Regex>	<Numeric>	<Numeric>
▼ javaString java.lang.String @ 0x6d2190ee8	24	48
> <class> class java.lang.String @ 0x6d2700980 System Class, Native Stack	24	56
> value byte[5] @ 0x6d2190f18 JAVAC	24	24
Σ Total: 2 entries		

```
String javaString = new String("JAVÁĎ");
```

UTF-16

J		A		V		A		Ć	
0x00	0x4A	0x00	0x41	0x00	0x56	0x00	0x41	0x01	0x06

Strings containing characters with non-zero upper byte cannot be compressed

➤ stored as 2 byte characters using UTF-16 encoding

UTF-16 String size constraints

```
new String(new char[<size>])    // UTF-16 String
```



```
// MAX_LENGTH = Integer.MAX_VALUE >> 1    ; e.g. 1,073,741,823
```

```
public static byte[] newBytesFor(int len) {
```

```
    // ...
```

```
    if (len > MAX_LENGTH) {
```

```
        throw new OutOfMemoryError("UTF16 String size is " + len +  
                                    ", should be less than " + MAX_LENGTH);
```

```
    }
```

```
    return new byte[len << 1];
```

```
}
```



UTF-16 Strings size should not be greater than **Integer.MAX_VALUE >> 1** due to underlying byte[] which is doubled. Not needed for LATIN-1 or w/o CompactStrings

String Construction

String Construction

- first, it try to compress the input `char[]` to `Latin-1` by stripping off upper bytes
- if it fails, `UTF-16` encoding is used (i.e. each char spreads across 2 bytes)

```
String(char[] value, int off, int len, Void sig) {  
    // ...  
    if (COMPACT_STRINGS) {  
        byte[] val = StringUTF16.compress(value, off, len);  
        if (val != null) {  
            this.value = val;  
            this.coder = LATIN1;  
            return;  
        }  
    }  
    this.coder = UTF16;  
    this.value = StringUTF16.getBytes(value, off, len);  
}
```

```
@Param({ "ĐŽĐ¹,Đ²ÑÑ`Đ" })  
public String utf_16_str1;
```

```
@Param({ "Đ¿Ñ€Đ¼Đ¿φĐ°χ" })  
public String utf_16_str2;
```

```
@Param({ "Đςζ»Đ¾,Ñ^ĐμÑ,, " })  
public String utf_16_str3;
```

```
@Benchmark
```

```
public String utf16_concat() {  
    return utf_16_str1 + utf_16_str2 + utf_16_str3 + utf_16_str4 + utf_16_str5;  
}
```

```
@Param({ "ΦÈ¾ẀχĐ»ĐΛρθÈ" })  
public String utf_16_str4;
```

```
@Param({ "ΩΔΘΞΨθςώςγθρ" })  
public String utf_16_str5;
```

```
@Param({ "ĐŽĐ¹,Đ²ÑÑ`Đ" })  
public String utf_16_str1;
```

```
@Param({ "Đ¿ÑĎĐ³Đ¿φĐ°X" })  
public String utf_16_str2;
```

```
@Param({ "Đςζ»Đ¾,Ñ^ĐµÑ,, " })  
public String utf_16_str3;
```

```
@Benchmark
```

```
public String utf16_concat() {  
    return utf_16_str1 + utf_16_str2 + utf_16_str3 + utf_16_str4 + utf_16_str5;  
}
```

```
@Param({ "ΦÈ¾ẀXĐ»ĐΛρ8Ě" })  
public String utf_16_str4;
```

```
@Param({ "ΩΔΘΞΨΘςώςγ8ρ" })  
public String utf_16_str5;
```

	ns/op	B/op
-XX:+CompactStrings	44.4	168
-XX:-CompactStrings	35.7	168

```
@Param({ "ĐŽĐ¹,Đ²ÑÑ`Đ" })  
public String utf_16_str1;
```

```
@Param({ "ФЁ³ẀΧĐ»ΔΛρøË" })  
public String utf_16_str4;
```

```
@Param({ "Đ¿ÑĖĐ³Đ¿φĐ°Χ" })  
public String utf_16_str2;
```

```
@Param({ "ΩΔΘΞΨθςώςγδρ" })  
public String utf_16_str5;
```

```
@Param({ "Đςζ»Đ¾,Ñ^ĐμÑ,, " })  
public String utf_16_str3;
```

```
@Benchmark  
public String utf16_concat() {  
    return utf_16_str1 + utf_16_str2 + utf_16_str3 + utf_16_str4 + utf_16_str5;  
}
```

	ns/op	B/op
-XX:+CompactStrings	44.4	168
-XX:-CompactStrings	35.7	168



For applications that extensively use UTF-16 characters, consider disabling Compact Strings for a better performance.

byte → char

byte[] → char[]

Inflation: byte → char or byte[] → char[]

- whenever a `char` or `char[]` representation is needed from a `LATIN-1` encoded String (e.g. `charAt()`, `toCharArray()`)

LATIN-1

```
public static char charAt(byte[] value, int index) {  
    // ...  
    return (char) (value[index] & 0xff);  
}
```

UTF-16

```
@HotSpotIntrinsicCandidate  
// intrinsic performs no bounds checks  
static char getChar(byte[] val, int index) {  
    // ...  
    index <= 1;  
    return (char) (((val[index++] & 0xff) << HI_BYTE_SHIFT) |  
                   ((val[index] & 0xff) << LO_BYTE_SHIFT));  
}
```

```
@Param({ "17" })  
public int idx;
```

```
@Param({ "123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ" }) // length 35  
public String latin_1;
```

```
@Param({ "123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ" }) // length 35  
public String utf_16;
```

```
@Benchmark  
public char[] latin1_toCharArray() {  
    return latin_1.toCharArray();  
}
```

```
@Benchmark  
public char[] utf16_toCharArray() {  
    return utf_16.toCharArray();  
}
```

```
@Benchmark  
public char latin1_charAt() {  
    return latin_1.charAt(idx);  
}
```

```
@Benchmark  
public char utf16_charAt() {  
    return utf_16.charAt(idx);  
}
```

	JDK 9.0.4	JDK 8u172
	ns/op	ns/op
latin1_charAt	3.3	2.9
utf16_charAt	3.0	2.9
latin1_toCharArray	11.9	11.7
utf16_toCharArray	12.5	11.7

Inflating from byte → char or byte[] → char[] has minimal performance impact for this test case!

String.equals()

String.equals()

➤ **equals()** does not actually touches the **value** if **coders** differ!

```
public boolean equals(Object anObject) {  
    // ...  
    if (anObject instanceof String) {  
        String aString = (String)anObject;  
        if (coder() == aString.coder()) {  
            return isLatin1() ? StringLatin1.equals(value, aString.value)  
                : StringUTF16.equals(value, aString.value);  
        }  
    }  
    return false;  
}
```

```
@Param({ "123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ" })  
public String latin_1;
```

```
@Param({ "123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ" })  
public String latin_1_identical;
```

```
@Param({ "123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ³¼½" })  
public String utf_16;
```

```
@Param({ "123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ³¼½" })  
public String utf_16_identical;
```

```
@Benchmark  
public boolean latin1_toLatin1_equals() {  
    return latin_1.equals(latin_1_identical);  
}
```

```
@Benchmark  
public boolean utf16_toUtf16_equals() {  
    return utf_16.equals(utf_16_identical);  
}
```

```
@Benchmark  
public boolean latin1_toUtf16_equals() {  
    return latin_1.equals(utf_16);  
}
```

JDK 9 defaults

	-XX:+CompactStrings	-XX:-CompactStrings
	ns/op	ns/op
latin1_toLatin1_equals	5.9	6.5
utf16_toUtf16_equals	6.4	6.5
latin1_toUtf16_equals	2.8	6.4

-XX:+CompactStrings

- if coders differ, Strings comparison is cheap!
- if same coder, slightly better performance for Latin-1

-XX:-CompactStrings

- almost the same performance in all cases



How will this be compiled at Bytecode level (i.e. JDK 9)?

```
private static String getMessage() {  
  
    String str1 = " Java";  
    String str2 = " conference";  
  
    return str1 + str2;  
}
```

```
private static String getMessage() LString;
```

```
LDC " Java"
```

```
ASTORE 0
```

```
LDC " conference"
```

```
ASTORE 1
```

```
ALOAD 0
```

```
ALOAD 1
```

```
INVOKEDYNAMIC makeConcatWithConstants(LString;LString;)LString; [
```

```
    // handle kind 0x6 : INVOKESTATIC
```

```
    StringConcatFactory.makeConcatWithConstants(...) LCallSite;
```

```
    // arguments:
```

```
    "\u0001\u0001"
```

```
]
```

```
ARETURN
```

```
private static String getMessage() LString;
```

```
LDC " Java"
```

```
ASTORE 0
```

```
LDC " conference"
```

```
ASTORE 1
```

```
ALOAD 0
```

```
ALOAD 1
```

```
INVOKEDYNAMIC makeConcatWithConstants(LString;LString;)LString; [
```

```
    // handle kind 0x6 : INVOKESTATIC
```

```
    StringConcatFactory.makeConcatWithConstants(...) LCallSite;
```

```
    // arguments:
```

```
    "\u0001\u0001"
```

```
]
```

```
ARETURN
```



Built in optimized String concatenation handlers, implementable without any need to change the Java-to-bytecode compiler.

A tricky scenario

e.g. String concatenation in JDK 8 vs JDK 9

```
public class StringConcat {  
  
    public static void main(String[] args) {  
        Message message = new Message();  
        System.out.println("" + message + message);  
    }  
  
    static class Message {  
        String[] message = new String[] { " Java", " conference" };  
        int index = 0;  
  
        @Override  
        public String toString() {  
            return message[index++];  
        }  
    }  
}
```

```
public class StringConcat {  
  
    public static void main(String[] args) {  
        Message message = new Message();  
        System.out.println("" + message + message);  
    }  
  
    static class Message {  
        String[] message = new String[] { " Java", " conference" };  
        int index = 0;  
  
        @Override  
        public String toString() {  
            return message[index++];  
        }  
    }  
}
```



```
$ jdk-8/bin/java StringConcat  
Java conference
```



```
$ jdk-9/bin/java StringConcat  
conference Java
```

```
public class StringConcat {  
  
    public static void main(String[] args) {  
        Message message = new Message();  
        System.out.println("" + message + message);  
    }  
  
    static class Message {  
        String[] message = new String[] { " Java", " conference" };  
        int index = 0;  
  
        @Override  
        public String toString() {  
            return message[index++];  
        }  
    }  
}
```



```
$ jdk-8/bin/java StringConcat  
Java conference
```



```
$ jdk-9/bin/java -Djava.lang.invoke.stringConcat=BC_SB_SIZED_EXACT StringConcat  
Java conference
```

```
public class StringConcat {  
  
    public static void main(String[] args) {  
        Message message = new Message();  
        System.out.println("" + message + message);  
    }  
  
    static class Message {  
        String[] message = new String[] { " Java", " conference" };  
        int index = 0;  
  
        @Override  
        public String toString() {  
            r  
        }  
    }  
}
```

Any bytecode StringBuilder strategy works!
e.g. **BC_SB**, **BC_SB_SIZED**, **BC_SB_SIZED_EXACT**
Default strategy is **MH_INLINE_SIZED_EXACT**



```
$ jdk-8/bin/java StringConcat  
Java conference
```



```
$ jdk-9/bin/java -Djava.lang.invoke.stringConcat=BC_SB_SIZED_EXACT StringConcat  
Java conference
```



JDK / JDK-8200118

[TESTBUG] String concat tests should test toString() application order

Details

Type:	Enhancement	Status:	RESOLVED
Priority:	P4	Resolution:	Fixed
Affects Version/s:	None	Fix Version/s:	11
Component/s:	core-libs		
Labels:	CPU18_07-critical-SQE-OK jdk10u-fix-request		

Description

This issue covers the potential issue with MH_* strategies in StringConcatFactory that may end up calling toString() on arguments in wrong order. Non MH-strategies appear fine. See the evidence in the wild here:

<https://www.guardsquare.com/en/blog/string-concatenation-java-9-conversion-confusion>

This issue adds the relevant test for String concatenation. JDK 10 fails this new test. JDK 11+ passes the test, because the actual bug is [JDK-8194554](#), which is fixed. In other words, StringConcatFactory implementation is actually correct, and it expects filterArguments to run in the specified order. Selectively applying fix from [JDK-8194554](#) to JDK 10 makes it pass.

Stack Walker

JEP 259: Stack-Walking API

Stack-Walker - flexible mechanism to traverse and materialize the required stack-frame information allowing efficient lazy access to additional stack frames when required.

It avoids the cost of examining all frames if caller is only interested in a few.

Stack-Walker features:

- lazy frames construction
- limit stack depth
- filter frames

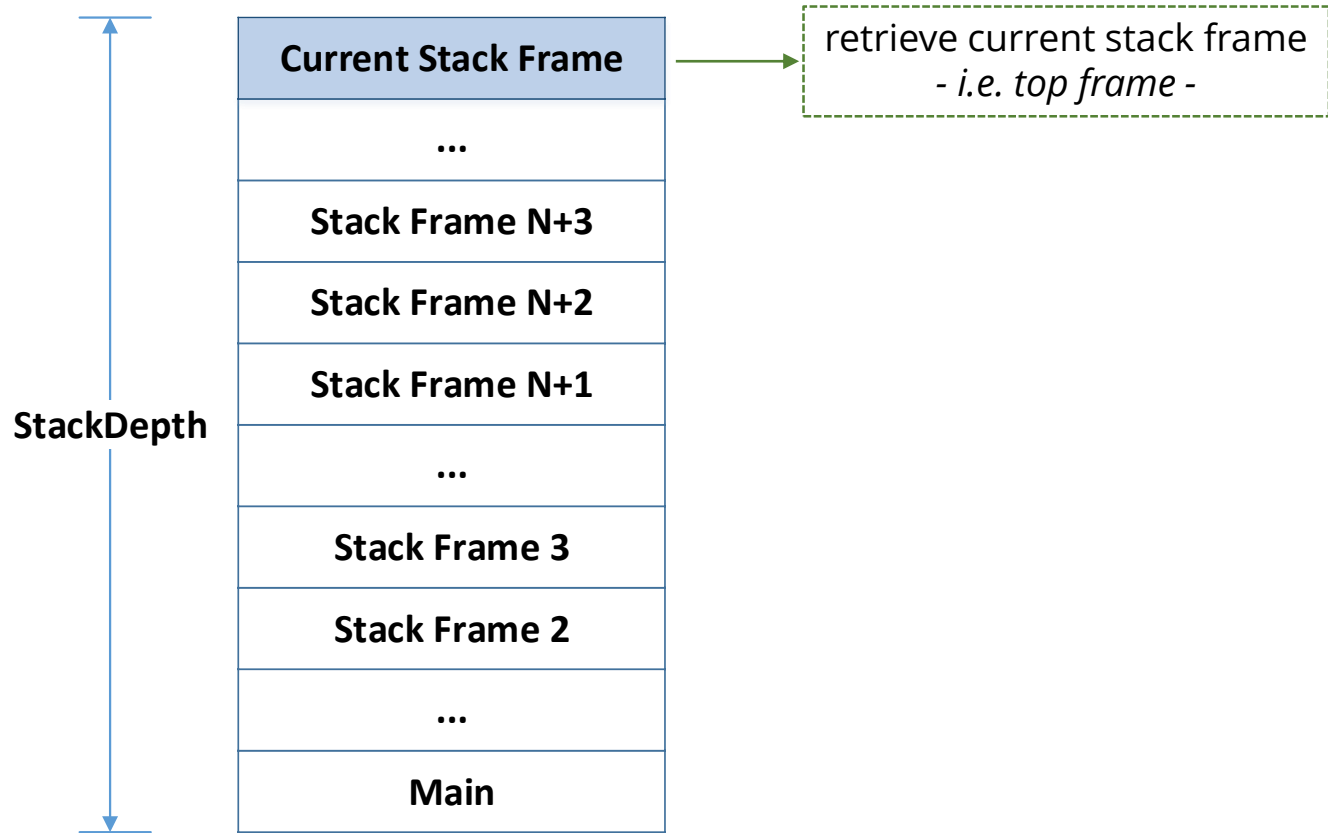
StackWalker **getInstance**()

StackWalker **getInstance**(Set<Option> options)

Option:

```
// support for getCallerClass() && getDeclaringClass()  
RETAIN_CLASS_REFERENCE  
// support reflexion frames for  
// reflect.Method#invoke && reflect.Constructor#newInstance  
SHOW_REFLECT_FRAMES  
// support for all hidden frames  
SHOW_HIDDEN_FRAMES
```

StackWalker **getInstance**(Set<Option> options, **int** estimateDepth)



Retrieve Current Stack Frame - Stack Walker

@Benchmark

```
public StackWalker.StackFrame retrieveCurrentStackFrame() {  
    return recCurrentStackFrame(stackDepth);  
}  
  
private StackWalker.StackFrame recCurrentStackFrame(int depth) {  
    if (depth == 0) {  
        return getCurrentStackFrame();  
    }  
    return recCurrentStackFrame(depth - 1);  
}  
  
private StackWalker.StackFrame getCurrentStackFrame() {  
    return StackWalker.getInstance()  
        .walk(stream -> stream.findFirst())  
        .orElseThrow(NoSuchElementException::new);  
}
```



Retrieve Current Stack Frame - `getStackTrace()`

```
private StackTraceElement getCurrentStackFrame() {  
    StackTraceElement[] stackTrace = Thread.currentThread().getStackTrace();  
    return Arrays.stream(stackTrace)  
        .findFirst()  
        .orElseThrow(NoSuchElementException::new);  
}
```

Retrieve Current Stack Frame

StackDepth	1	10	100	1000
	us/op	us/op	us/op	us/op
StackWalker	1.7	1.8	1.9	3.4
getStackTrace()	13.2	17.8	59.3	470.2

StackWalker - offers almost constant access to current top frame for different StackDepth sizes

getStackTrace() - cost seems proportional to StackDepth and is around one order of magnitude slower, or even worse

Performance scenario:

**process asynchronously throwable
frames (i.e. in another thread)**

vs.

synchronously stack walking

↓
Thread #1
↓

```
Throwable throwable ← new Throwable();  
// do other stuff
```

≈ asynch

↓
Thread #2
↓

```
throwable.printStackTrace();  
// process asynchronously stack frames
```

↓
Thread #1
↓

```
Throwable throwable ← new Throwable();  
// do other stuff
```

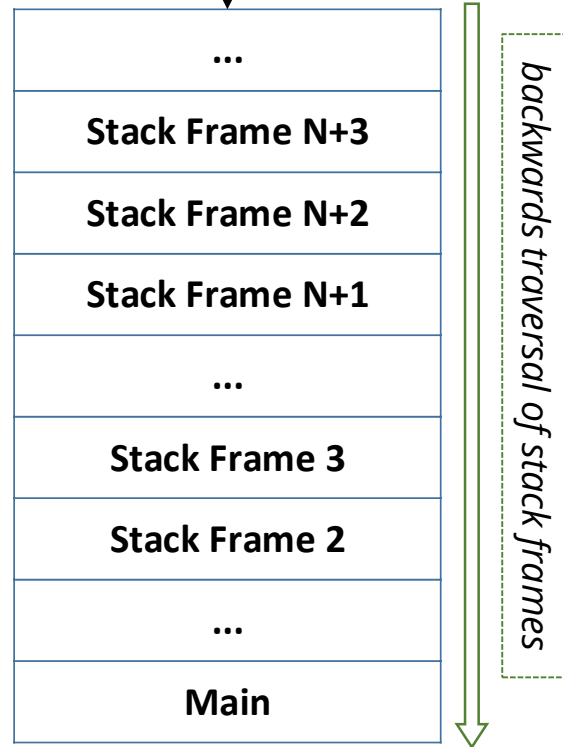
≈ asynch

↓
Thread #2
↓

```
throwable.printStackTrace();  
// process asynchronously stack frames
```

VS.

↓
Thread
↓



StackDepth	1	10	100	1000
	us/op	us/op	us/op	us/op
StackWalker ^[1]	1.7	5.4	33.3	307.9
new Throwable() ^[2]	1.2	1.6	6.1	49.8

Processing asynchronously throwable frames (i.e. in another thread) is faster than even using Stack Walker API!

Leverage to this mechanism for affordable cases!

^[1] – cost of synchronously traverse backwards stack frames

^[2] – exclude the cost of asynchronously generating stack trace elements (i.e. `getStackTrace()`)

Collection Factory Methods

JEP 269: Convenience Factory Methods for Collections

Factory Methods - static factory methods on the collection interfaces that will create compact, unmodifiable collection instances.

```
List.of()           //e.g. List.of("a", "b", "c");
```

```
Set.of()            //e.g. Set.of("a", "b", "c");
```

```
Map.of()            //e.g. Map.of("a", "v1", "b", "v2")
```

```
Map.ofEntries()     //e.g. Map.ofEntries(entry("a", "v1"), entry("b", "v2"))
```

List.of()

fixed args for ≤ 10 elements

```
<E> List<E> of()  
<E> List<E> of(E e1)  
<E> List<E> of(E e1, E e2)  
<E> List<E> of(E e1, E e2, E e3)  
<E> List<E> of(E e1, E e2, E e3, E e4)  
<E> List<E> of(E e1, E e2, E e3, E e4, E e5)  
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6)  
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)  
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)  
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)  
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)  
  
<E> List<E> of(E... elements) // varargs
```



Why are there **of()** methods explicitly declared for first 10 params?

fixed args for ≤ 10 elements

```
<E> List<E> of()
<E> List<E> of(E e1)
<E> List<E> of(E e1, E e2)
<E> List<E> of(E e1, E e2, E e3)
<E> List<E> of(E e1, E e2, E e3, E e4)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)

<E> List<E> of(E... elements) // varargs
```

`var_args(int ... args) method call` ; 4 params

prior calling `var_args`

```
mov    DWORD PTR [rdx+0x8],0xf800016d    ;{metadata({type array int})}  
  
mov    r10d,DWORD PTR [rbp+0x10]         ;*getfield param2  
mov    r8d,DWORD PTR [rbp+0x14]         ;*getfield param3  
mov    r11d,DWORD PTR [rbp+0x18]         ;*getfield param4  
mov    ecx,DWORD PTR [rbp+0xc]           ;*getfield param1  
  
mov    DWORD PTR [rdx+0x10],ecx          ;*iastore  
mov    DWORD PTR [rdx+0x14],r10d        ;*iastore  
mov    DWORD PTR [rdx+0x18],r8d         ;*iastore  
mov    DWORD PTR [rdx+0x1c],r11d        ;*newarray
```

```
call    0x00007f68e963e8c0              ;*invokespecial var_args
```

`var_args` method call with 4 params

```
mov     DWORD PTR [rdx+0x8],0xf800016d    ;{metadata({type array int})}
```

```
p      6+1  DWORD PTR [rdx+0+1001]  ;
```



varargs method calls deal with extra stack parameter manipulation and array allocation! (i.e. slowing down the performance)

```
mov     DWORD PTR [rdx+0x10],ecx          ;*iastore
```

```
mov     DWORD PTR [rdx+0x14],r10d        ;*iastore
```

```
mov     DWORD PTR [rdx+0x18],r8d         ;*iastore
```

```
mov     DWORD PTR [rdx+0x1c],r11d        ;*newarray
```

```
call    0x00007f68e963e8c0              ;*invokespecial var_args
```

	explicit method params	varargs method params
	ns/op	ns/op
2_params	2.3	4.3
4_params	2.5	4.9
6_params	3.0	7.0
8_params	3.3	7.9
10_params	3.7	8.6

Declaring methods with explicit number of parameters leads to better performance in comparison to using analogous varargs method!

```
static <E> List<E> of() {  
    return ImmutableCollections.List0.instance();  
}
```

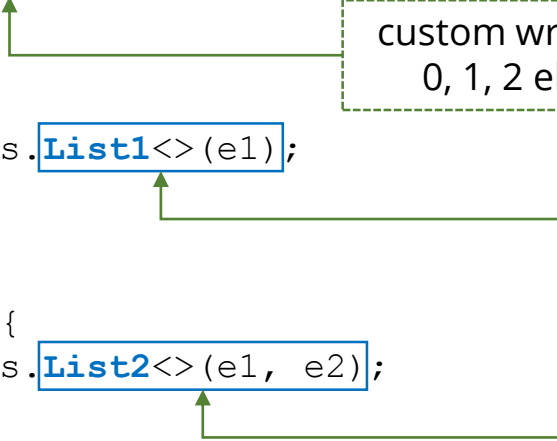
```
static <E> List<E> of(E e1) {  
    return new ImmutableCollections.List1<>(e1);  
}
```

```
static <E> List<E> of(E e1, E e2) {  
    return new ImmutableCollections.List2<>(e1, e2);  
}
```

```
static <E> List<E> of(E e1, E e2, E e3) {  
    return new ImmutableCollections.ListN<>(e1, e2, e3);  
}
```

```
static <E> List<E> of() {  
    return ImmutableCollections.List0.instance();  
}  
  
static <E> List<E> of(E e1) {  
    return new ImmutableCollections.List1<>(e1);  
}  
  
static <E> List<E> of(E e1, E e2) {  
    return new ImmutableCollections.List2<>(e1, e2);  
}  
  
static <E> List<E> of(E e1, E e2, E e3) {  
    return new ImmutableCollections.ListN<>(e1, e2, e3);  
}
```

custom wrappers for
0, 1, 2 elements



generic wrapper for N > 2 elements


```
static final class List0<E> extends AbstractImmutableList<E> {  
    private static final List0<?> INSTANCE = new List0<>();  
    //...  
}
```

```
static final class List1<E> extends AbstractImmutableList<E> {  
    private final E e0;  
    //...  
}
```

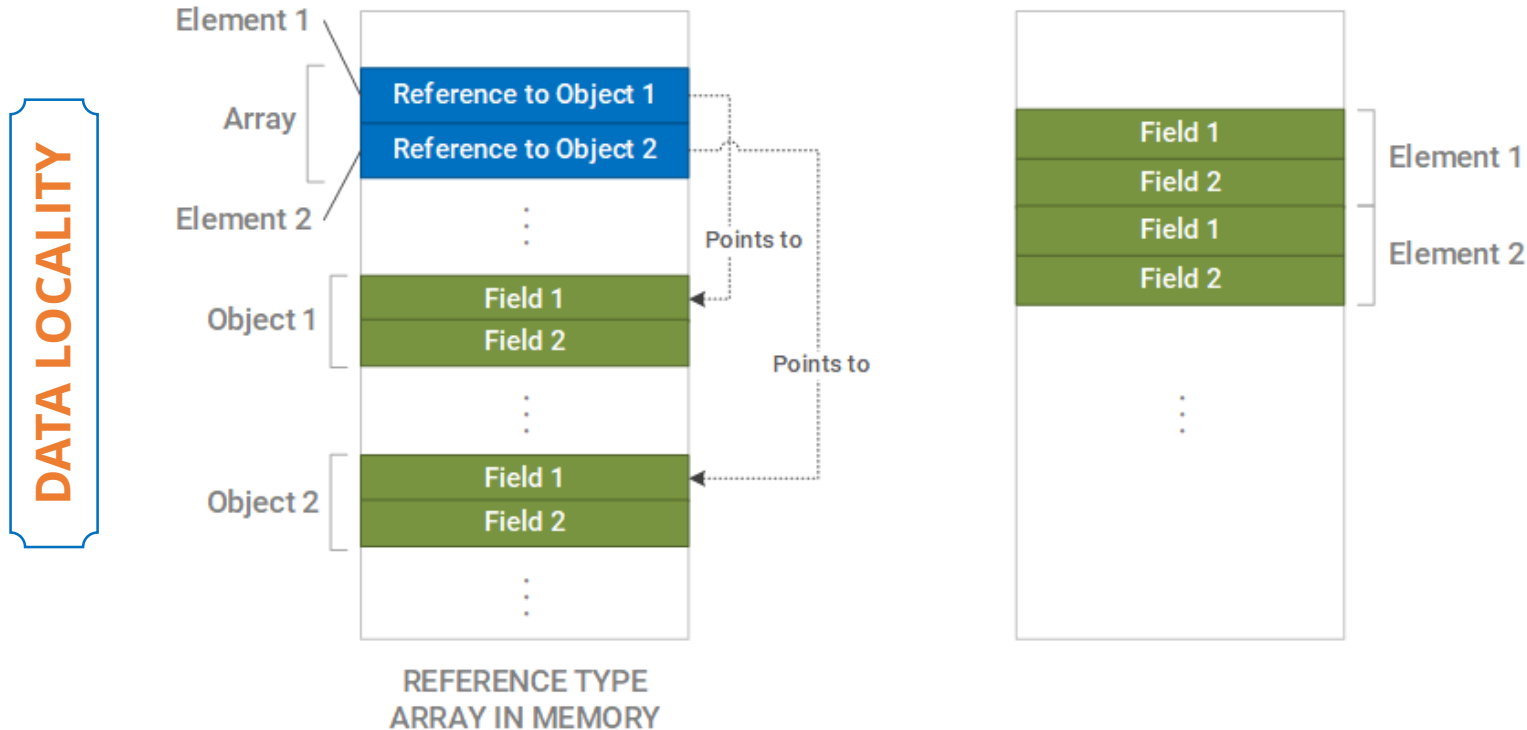
```
static final class List2<E> extends AbstractImmutableList<E> {  
    private final E e0;  
    private final E e1;  
    //...  
}
```

```
static final class ListN<E> extends AbstractImmutableList<E> {  
    private final E[] elements;  
    //...  
}
```

field based
implementations

array based
implementation

Packing fields vs. using an array of elements



source: [<https://dzone.com/articles/what-is-project-Valhalla>]

Collections.unmodifiableList() vs. List.of()

JDK 8

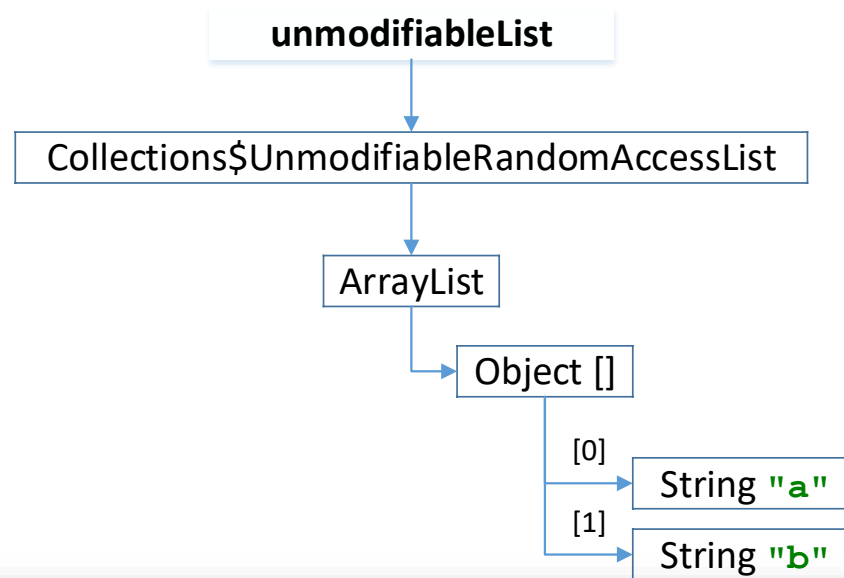
```
List<String> arrayList = new ArrayList<>();  
arrayList.add("a");  
arrayList.add("b");  
List<String> unmodifiableList = Collections.unmodifiableList(arrayList);
```

VS.

JDK 9

```
List<String> listOf = List.of("a", "b");
```

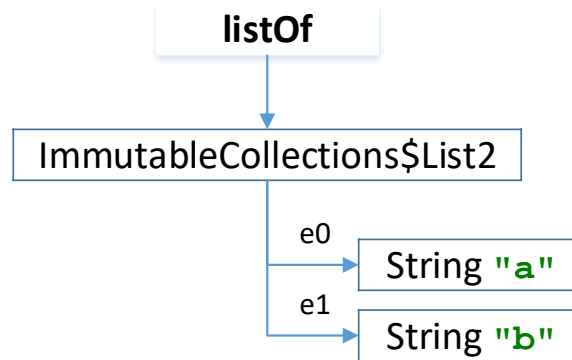
JDK 8



Class Name	Shallow Heap	Retained Heap
▼  arrayList java.util.ArrayList @ 0x6d2192598	24	80
>  <class> class java.util.ArrayList @ 0x6d2747880 System Class	24	40
>  elementData java.lang.Object[10] @ 0x6d21925e0	56	56
Σ Total: 2 entries		
▼  unmodifiableList java.util.Collections\$UnmodifiableRandomAccessList @ 0x6d2192770	24	24
>  <class> class java.util.Collections\$UnmodifiableRandomAccessList @ 0x6d23ebb48 System Class	8	8
>  list, c java.util.ArrayList @ 0x6d2192598	24	80
Σ Total: 2 entries		

Total retained heap: 80 + 24 bytes

JDK 9



Class Name	Shallow Heap	Retained Heap
listOf java.util.ImmutableCollections\$List2 @ 0x6d2192788	24	24
<class> class java.util.ImmutableCollections\$List2 @ 0x6d274d990 System Class	0	0
e0 java.lang.String @ 0x6d21925b0	24	48
e1 java.lang.String @ 0x6d2192618	24	48
Σ Total: 3 entries		
Σ Total: 4 entries	Total retained heap: 24 bytes	

# of elements	ns/op	
	of()	unmodifiableList()
empty	2.6	7.6
1	4.8	16.8
2	5.9	20.7
3	12.5	23.8
10	25.9	48.0
11	27.4	61.5

`List.of()` offers better performance in comparison to `Collections.unmodifiableList()`

SUM UP

Less overall space

Improved reference locality

Faster creation time

Set.of()

fixed args for ≤ 10 elements

```
<E> Set<E> of()  
<E> Set<E> of(E e1)  
<E> Set<E> of(E e1, E e2)  
<E> Set<E> of(E e1, E e2, E e3)  
<E> Set<E> of(E e1, E e2, E e3, E e4)  
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5)  
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6)  
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)  
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)  
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)  
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)  
  
<E> Set<E> of(E... elements) // varargs
```

```
static final class Set0<E> extends AbstractImmutableSet<E> {...}  
static final class Set1<E> extends AbstractImmutableSet<E> {...}  
static final class Set2<E> extends AbstractImmutableSet<E> {...}  
static final class SetN<E> extends AbstractImmutableSet<E> {...}
```

# of elements	ns/op	
	of()	unmodifiableSet()
empty	2.6	10.4
1	4.0	25.6
2	8.0	40.5
3	42.5	51.6
10	100.9	128.3
11	110.9	139.5

`Set.of()` offers better performance in comparison to `Collections.unmodifiableSet()`

Map.of()

fixed args for ≤ 10 key-values

```
<K, V> Map<K, V> of()
<K, V> Map<K, V> of(K k1, V v1)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7, K k8, V v8)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7, K k8, V v8, K k9, V v9)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7, K k8, V v8, K k9, V v9, K k10, V v10)
```

```
<K, V> Map<K, V> ofEntries(Entry<? extends K, ? extends V>... entries) // varargs
```

Map Wrappers

```
static final class Map0<K, V> extends AbstractImmutableMap<K, V> {...}
```

```
static final class Map1<K, V> extends AbstractImmutableMap<K, V> {...}
```

```
static final class MapN<K, V> extends AbstractImmutableMap<K, V> {...}
```

# of elements	ns/op		
	of()	unmodifiableMap()	ofEntries()
empty	2.6	9.1	2.5
1	6.0	25.0	9.4
2	39.1	40.1	55.9
3	48.3	48.0	78.5
4	64.8	61.2	99.7
10	154.9	129.7	249.7
11	-	141.0	272.4

`Map.of()` offers better performance in comparison to `Map.ofEntries()`, however in few cases it seems equivalent to `Collections.unmodifiableMap()` (e.g. for 2, 3, 4, 10 elements)

`Map.ofEntries()` behaves slower than the others (i.e. varargs might be a potential cause)

onSpinWait()

JEP 285: Spin-Wait Hints

`onSpinWait()` - tells the CPU there is busy-waiting loop that may burn few CPU-cycles waiting for something to happen.

CPU can assign more resources to other threads, without actually loading the OS scheduler and dequeuing another thread (which may be expensive).

`onSpinWait()` **example usages** - for events which might happen very frequently (i.e. high rate) and finish very quickly

```
// busy spin until condition is satisfied  
while (!condition_not_satisfied) {  
    Thread.onSpinWait();  
}
```

Scenario

↓
Producer
↓

```
for (long i = 0; i < total_items; i++) {  
    while (!ready_to_produce()) {  
        Thread.onSpinWait();  
    }  
    produce_item();  
}  
signal_finish(); // is_running <- false
```

↓
Consumer
↓

```
while (is_running) {  
    while (!ready_to_consume()) {  
        Thread.onSpinWait();  
    }  
    consume();  
}
```

Thread.onSpinWait()



PAUSE

```
;*invokestatic onSpinWait()
```



x86 hotspot intrinsic



PAUSE instruction delays the next instruction's execution for a finite period of time, hence parts of the pipeline are no longer being used → reduces *power consumption*!

source: [<https://software.intel.com/en-us/articles/benefitting-power-and-performance-sleep-loops>]

	us/op	context-switches
onSpinWait()	47.1	207
yield()	143.4	220
sleep(1)	1,182,968.2	5,640

Experiment: looping 1024 iterations, inside each iteration calling onSpinWait() / yield() / sleep(1)



JDK / JDK-8159532

Investigate SPARC intrinsic for Thread.onSpinWait

Agile Board

Details

Type:	Enhancement	Status:	OPEN
Priority:	P4	Resolution:	Unresolved
Affects Version/s:	9, 10	Fix Version/s:	tbd_feature
Component/s:	hotspot		
Labels:	bleaklow-interest	c1	c2 c2-intrinsic performance
Subcomponent:	compiler		
CPU:	sparc		

Description

[JDK-8147844](#) implements Thread.onSpinWait and the correspond x86 intrinsic.

Investigate whether it's possible to support an intrinsic method on SPARC.

Contended locks in Java 9

JEP 143: Improve Contended Locking

Improve the overall performance of contended Java object monitors.

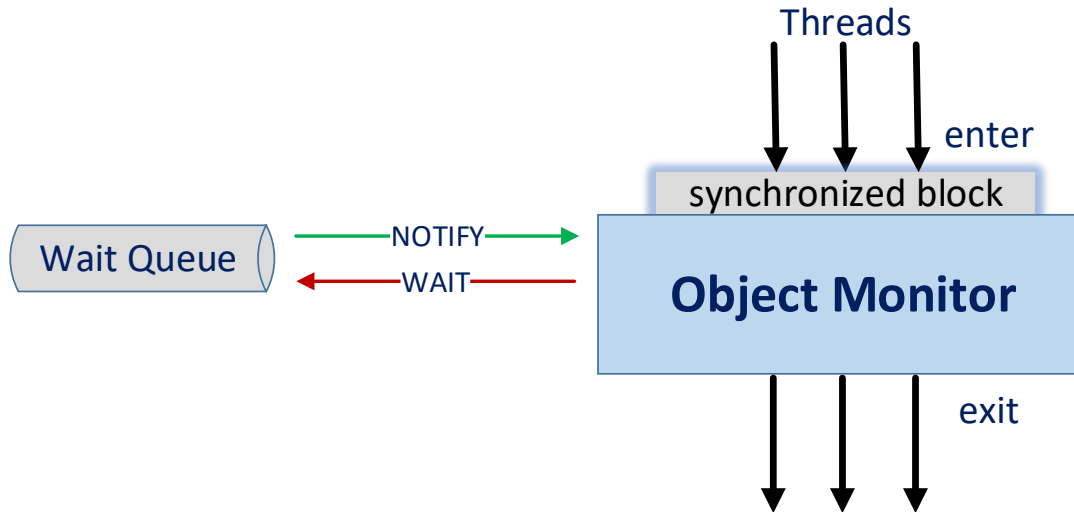
Contended Lock - a thread wants to enter the synchronized method that is already locked by another thread

Few related areas:

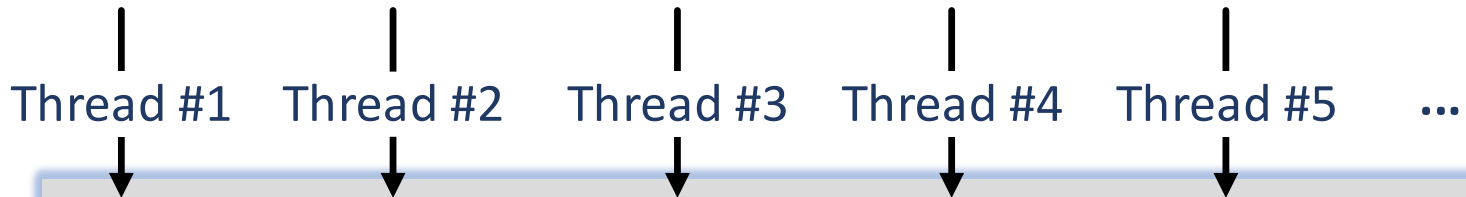
- Fast Java monitor enter/exit operations
- Fast Java monitor **notify/notifyAll** operations
- Speed up **PlatformEvent::unpark()**
- Field reordering and cache line alignment

Every Object has an associated:

- monitor
- “WaitSet” for waiting threads (i.e. heavy weight locking scheme)



```
int loop_count = 50;  
volatile long consumeCPU = 0;
```



```
public synchronized void compute(int loop_count)  
{  
    for (long i = loop_count; i > 0; i--) {  
        consumeCPU += ++consumeCPU & 0xFF // "heavy" processing  
    }  
}
```

# of threads	us/op	
	-XX:-UseBiasedLocking -XX:+UseHeavyMonitors	JDK 9 defaults -XX:+UseBiasedLocking -XX:-UseHeavyMonitors
1	0.4	0.3
2	2.1	1.7
4	4.2	3.9
8	18.2	10.7
16	4,284.7	1,133.6
32	9,996.6	5,362.9
64	31,627.9	35,048.4
128	50,096.6	52,024.7
256	74,472.8	99,485.1
512	184,838.9	218,215.3
1024	189,070.8	505,467.4

Contended locks offers significant better performance when the number of contended threads is significant higher (i.e. 1024)!

For relatively small contention, JDK default parameters (i.e. w/o heavy monitors) seems more appropriate

THANK YOU



Krakow, 9-11 May 2018

Ionuț Baloșin

Software Architect

 @ionutbalosin

Bibliography

Compact Strings - Aleksey Shipilëv

- ♦ <https://www.youtube.com/watch?v=wlyeOaitmWM>

Java 9: Beyond Contention! - Monica Beckwith

- ♦ https://www.youtube.com/watch?v=q1CeN_mR-II

Collections Refueled by Stuart Marks

- ♦ <https://www.youtube.com/watch?v=LI8rpkOGU3c>

Translation of Lambda Expressions - Brian Goetz

- ♦ <http://cr.openjdk.java.net/~briangoetz/lambda/lambda-translation.html>

Everything you wanted to know about Stack Traces - Andrei Pangin

- ♦ <https://www.youtube.com/watch?v=FTsAXJdrJbl>

<https://www.guardsquare.com/en/blog/string-concatenation-java-9-conversion-confusion>

THANK YOU



Krakow, 9-11 May 2018

Ionuț Baloșin

Software Architect

 @ionutbalosin

APPENDIX

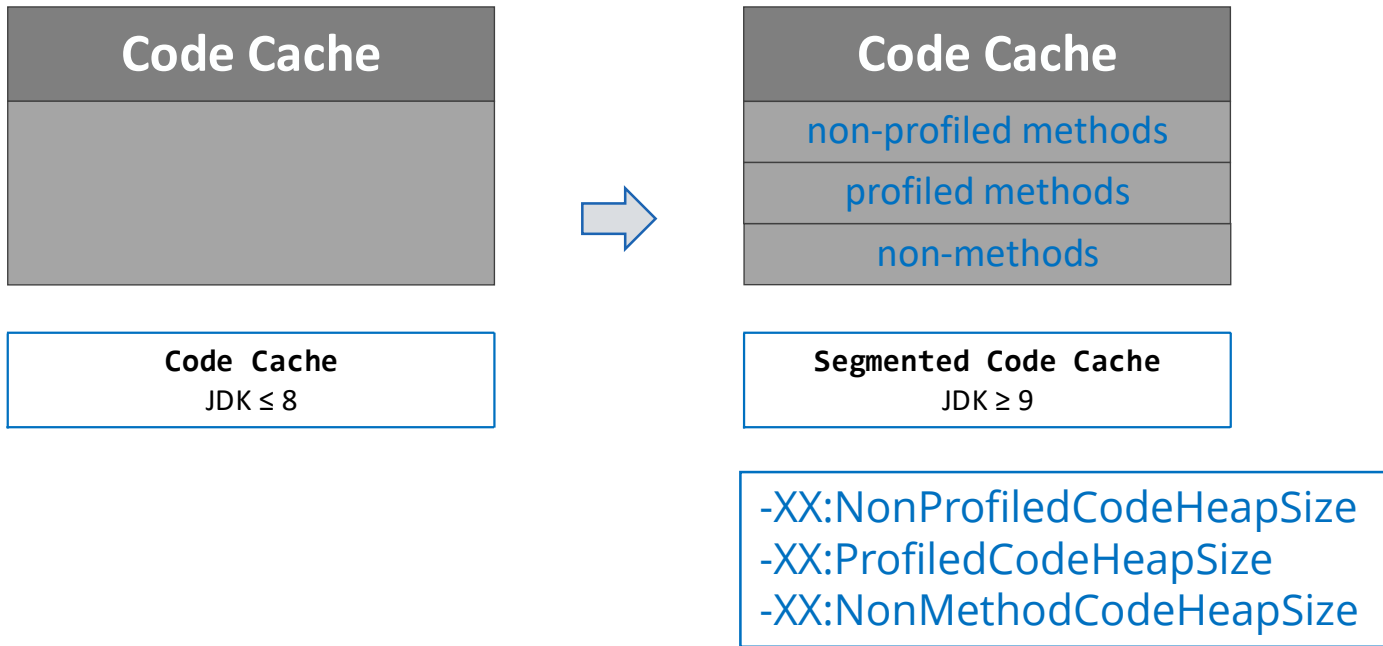


Krakow, 9-11 May 2018

Ionuț Baloșin

Software Architect
 @ionutbalosin

Segmented Code Cache



- **non-profiled** - fully optimized with a potentially long lifetime (C2 compiled)
- **profiled** - lightly optimized, profiled methods with a short lifetime (C1 compiled)
- **non-method** - VM internals (e.g. compiler buffers, bytecode interpreter)

Better code layout → improves performance (e.g. 9% speedup)

- reduces fragmentation
- less Sweeper overhead
- minimizes iTLB misses (e.g. 44% less iTLB misses)
- minimizes iCache misses (e.g. 14% less iCache misses)

source: [[The Java HotSpot VM Under the Hood – Tobias Hartmann & Zoltán Majó](#)]



Ionut Balosin @ionutbalosin · May 4

Hi @stuartmarks, regarding your talk Collections Refueled, I would like to ask if there is any particular reason to not have as well List3, List4 ... / Set3, Set4 ... / Map2 ..., etc (i.e. dedicated wrappers for more than 2/3 elements in case of Set/List/Map)? Any clue please?



1



1



Stuart Marks

@stuartmarks

Replying to @ionutbalosin

No single reason; a bunch of tradeoffs. More code to maintain or generate; diminishing space savings for the larger sizes; increased likelihood of slowdown due to megamorphic call sites.

2:31 PM - 4 May 2018

CallSite LambdaMetafactory.**metafactory()**

CallSite **buildCallSite()**

Class<?> innerClass = **spinInnerClass()**

*/** generates a class file which implements the functional interface */*
LambdaMain\$\$Lambda\$49.class

*/** if requested, dump out to a file for debugging purposes */*
-Djdk.internal.lambda.dumpProxyClasses=<path>

*/** creates and returns a call site having the target Method Handle a reference to a proxy of a inner generated class */*

MH ← **LambdaMain\$\$Lambda\$49/1864350231 .get\$Lambda()**
return callSite ← **new ConstantCallSite(MH)**

*/** links invokedynamic to target method implementation (e.g. **get\$Lambda**) which basically returns an instance of the functional interface (~ lambda object) */*

callSite.dynamicInvoker()
Invokers.linkToTargetMethod(type)

A Bit of History ...

UseCompressedStrings - introduced by Java 6 u 21

- an optional and experimental, non open source feature
- tried to compress **char[]** into **byte[]** on String construction, hence it did most String operations on char[], which required a lot of unpacking from byte[] to char[] since most of Strings could fit on byte (e.g. LATIN-1)
- feature implemented in **alt-rt.jar**. Optional features are harder to test, since they double the number of option combinations to try.

Source [<https://www.infoq.com/news/2016/02/compact-strings-java-jdk9>]

A Bit of History ...

UseCompressedStrings - introduced by Java 6 u 21

➤ an optional and experimental, non open source feature



Let's reuse the same idea however instead of switching between `byte[]` ↔ `char[]` to have String directly backed by `byte[]` whenever possible

➤ feature implemented in **alt-rt.jar**. Optional features are harder to test, since they double the number of option combinations to try.

Source [<https://www.infoq.com/news/2016/02/compact-strings-java-JDK9>]

A Bit of History ...

OptimizeStringConcat - introduced by Java 6 u 18

➤ fragile feature leading the code to fall outside the **AST** sometimes

```
private int counter;
```

```
@Benchmark
public String inlineSideEffect() {
    return new StringBuilder().append(counter++).toString();
}
```

JDK 8u5

2,2 ns/op

```
@Benchmark
public String spliceSideEffect() {
    int cnt = counter++;
    return new StringBuilder().append(cnt).toString();
}
```

1,1 ns/op

Source [<https://bugs.openjdk.java.net/browse/JDK-8043677>]

JDK ≤ 8

Old APIs require VM to eagerly capture entire stack snapshot / full stack trace (i.e. performance penalty)

`Throwable::getStackTrace`

`Thread::getStackTrace` - may return a partial stack (i.e. performance reasons ^[*])

`SecurityManager::getClassContext` - allows accessing actual instance of the class that declared the method represented by the stack frame

^[*] `-XX:+OmitStackTraceInFastThrow` - if an exception is re-thrown few times the method may be re-compiled and compiler may choose a faster tactic using pre-allocated exceptions omitting a stack trace

JDK 8

```
private static String getMessage() LString;
```

```
LDC " Java"
```

```
ASTORE 0
```

```
LDC " conference"
```

```
ASTORE 1
```

```
NEW StringBuilder
```

```
DUP
```

```
INVOKESPECIAL StringBuilder.<init> () V
```

```
ALOAD 0
```

```
INVOKEVIRTUAL StringBuilder.append (LString;) LStringBuilder;
```

```
ALOAD 1
```

```
INVOKEVIRTUAL StringBuilder.append (LString;) LStringBuilder;
```

```
INVOKEVIRTUAL StringBuilder.toString () LString;
```

```
ARETURN
```

JDK 8

```
private static String getMessage() LString;
```

```
LDC " Java"
```

```
ASTORE 0
```

```
LDC " conference"
```

```
ASTORE 1
```

```
NEW StringBuilder
```

```
DUP
```

```
INVOKESPECIAL StringBuilder.<init> () V
```

new StringBuilder()

Strings literals
concatenation

```
ALOAD 0
```

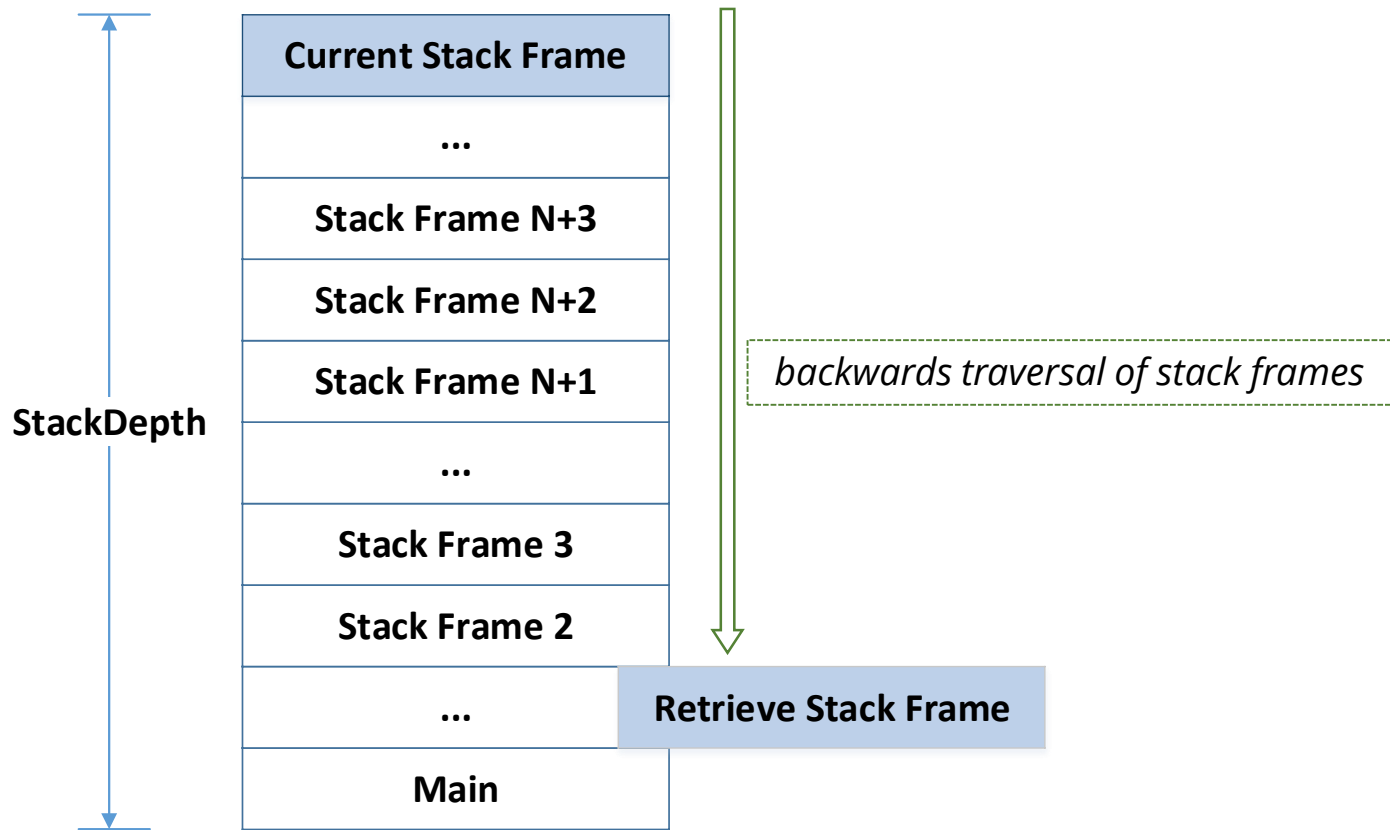
```
INVOKEVIRTUAL StringBuilder.append (LString;) LStringBuilder;
```

```
ALOAD 1
```

```
INVOKEVIRTUAL StringBuilder.append (LString;) LStringBuilder;
```

```
INVOKEVIRTUAL StringBuilder.toString () LString;
```

```
ARETURN
```



Retrieve Nth Stack Frame - Stack Walker

@Benchmark

```
public StackWalker.StackFrame retrieveBackNthStackFrame() {  
    return recBackNthStackFrame(stackDepth);  
}
```

```
private StackWalker.StackFrame recBackNthStackFrame(int depth) {  
    if (depth == 0) {  
        return getBackNthStackFrame(stackDepth);  
    }  
    return recBackNthStackFrame(depth - 1);  
}
```

```
private StackWalker.StackFrame getBackNthStackFrame(int numberOfSkippedFrames) {  
    return StackWalker.getInstance()  
        .walk(stream -> stream.skip(numberOfSkippedFrames - 1).findFirst())  
        .orElseThrow(NoSuchElementException::new);  
}
```

Retrieve Nth Stack Frame - `getStackTrace()`

```
private StackTraceElement getBackNthStackFrame(int numberOfSkippedFrames) {  
    StackTraceElement[] stackTrace = Thread.currentThread().getStackTrace();  
    return Arrays.stream(stackTrace).skip(numberOfSkippedFrames - 1).findFirst()  
        .orElseThrow(NoSuchElementException::new);  
}
```

Retrieve Nth Stack Frame

StackDepth	1	10	100	1000
	us/op	us/op	us/op	us/op
StackWalker	1.7	5.4	33.3	307.9
getStackTrace()	13.4	17.5	59.9	473.0

StackWalker offers better performance for traversing few stack frames, however it might become costly for bigger a StackDepth

```
@Benchmark
public Throwable justThrowException() {
    return recThrowException(stackDepth);
}

private Throwable recThrowException(int depth) {
    if (depth == 0) {
        return new Throwable();
    }
    return recThrowException(depth - 1);
}
```